API — Voz Contra o Fogo

Documentação Técnica Completa

IFPI — Campus Paulistana · FAPEPI · Pesquisa de Iniciação Científica

Versão 1.0 · 29/10/2026

---

1. O que é o Projeto

O **Voz Contra o Fogo** é uma plataforma de monitoramento participativo de queimadas desenvolvida pelo IFPI — Campus Paulistana em parceria com a FAPEPI. O projeto nasce da necessidade de ampliar a capacidade de resposta às queimadas na região da Caatinga do semiárido piauiense, especificamente nos municípios de **Paulistana, Jaicós e Acauã (PI)**.

A plataforma é composta por dois componentes principais:

- **Aplicativo móvel Flutter** — utilizado pela população para registrar denúncias de incêndios, visualizar focos ativos no mapa e acompanhar o status das ocorrências.
- **API Django (este documento)** — backend responsável por receber, processar, priorizar e distribuir as informações às autoridades ambientais.

2. Objetivos

Objetivo Geral

Desenvolver e disponibilizar uma API REST que centralize o recebimento de denúncias de queimadas, automatize a priorização por grau de risco e facilite a resposta coordenada das autoridades ambientais.

Objetivos Específicos

- Prover autenticação segura via JWT para cidadãos e autoridades com perfis distintos.
- Receber ocorrências com geolocalização, nível de fogo, fumaça e proximidade de residências.
- Calcular automaticamente um **score de prioridade** para cada ocorrência com base em múltiplos fatores de risco.
- Detectar **denúncias duplicadas** por proximidade geográfica e janela de tempo, evitando redundância.
- Notificar autoridades em tempo real via **FCM (Firebase Cloud Messaging)** a cada nova ocorrência.
- Transmitir atualizações de status via **WebSocket** para o app móvel.
- Fornecer endpoints de gestão de recursos de combate a incêndio (viaturas, fontes d'água).
- Suportar registro de verificações de campo realizadas pelas autoridades.

3. Stack Tecnológica

A API foi construída integralmente em Python com o framework Django, priorizando ferramentas consolidadas, open-source e adequadas ao contexto de pesquisa acadêmica.

Tecnologia	Versão	Finalidade
Python	3.11+	Linguagem principal
Django	4.2	Framework web MVC
Django REST Framework	3.15	Construção dos endpoints REST
SimpleJWT	5.3	Autenticação via tokens JWT
Django Channels	4.1	WebSocket para tempo real
Daphne (ASGI)	4.1	Servidor assíncrono para produção
PostgreSQL	—	Banco de dados em produção
SQLite	—	Banco de dados em desenvolvimento
Cloudinary	1.40	Armazenamento de imagens das ocorrências
PyFCM	2.0	Push notifications via Firebase
python-decouple	3.8	Gestão de variáveis de ambiente

4. Arquitetura e Estrutura

Estrutura de Apps Django

O projeto segue a arquitetura modular do Django, com cada domínio de negócio isolado em seu próprio app:

App	Responsabilidade
accounts	Usuários, autenticação JWT, perfis (cidadão / autoridade), FCM token
ocorrencias	Denúncias de incêndio, score de prioridade, detecção de duplicatas
alertas	Alertas emitidos pelas autoridades para a população
recursos	Recursos de combate (viaturas, fontes d'água, equipamentos)
verificacoes	Verificações de campo realizadas pelas autoridades
notifications	Consumers WebSocket e integração FCM

Fluxo de uma Denúncia

1. Cidadão preenche o formulário no app Flutter e toca em 'Enviar Denúncia'.
2. App verifica conectividade: se offline, salva no Hive (banco local) para sincronizar depois.
3. Se online, envia POST para **/api/ocorrencias/** com JWT no cabeçalho.
4. API valida o token e identifica o usuário (ou registra como anônimo).

5. API chama **calcular_score()** para calcular prioridade com base em nível de fogo, fumaça e proximidade.
6. Ocorrência é salva no banco com o score calculado.
7. API dispara notificação push via FCM para todos os dispositivos de autoridade cadastrados.
8. Se Django Channels + Redis estiver ativo, transmite via WebSocket para o painel das autoridades.
9. Autoridade visualiza a ocorrência, atualiza o status (em análise / atendida).
10. App do cidadão recebe a atualização de status via WebSocket ou próxima sincronização.

Cálculo de Score de Prioridade

Cada ocorrência recebe automaticamente um score numérico que determina sua urgência. O algoritmo considera os seguintes fatores:

Fator	Peso / Lógica
Nível do fogo (baixo/médio/alto/crítico)	Escala crescente de 1 a 4
Presença de fumaça	Incremento adicional ao score
Proximidade de residências	Fator multiplicador de urgência
Latitude / Longitude	Reservado para integração futura com dados INPE

5. Modelos de Dados

Usuario

Estende AbstractBaseUser. Autenticação via e-mail.

Campo	Tipo	Descrição
id	UUID (PK)	Identificador único automático
email	EmailField (único)	Login do usuário
nome	CharField(200)	Nome completo
role	CharField	'cidadao' ou 'autoridade'
fcm_token	TextField	Token de push notification
is_active / is_staff	Boolean	Controles de acesso Django
created_at	DateTimeField	Data de cadastro (auto)

Ocorrencia

Campo	Tipo	Descrição
id	UUID (PK)	Identificador único automático
usuario	FK → Usuario	Autor (null se anônima)
anonima	Boolean	Oculta identidade do autor
latitude / longitude	Decimal(10,7)	Coordenadas GPS da ocorrência
endereco	CharField(500)	Endereço geocodificado (opcional)
descricao	TextField	Relato livre do cidadão
nivel_fogo	CharField	baixo | medio | alto | critico
nivel_fumaca	CharField	Intensidade da fumaça
nivel_risco	CharField	Classificação de risco adicional
ha_fumaca	Boolean	Indicador de fumaça
proximo_residencia	Boolean	Fogo perto de casas
status	CharField	pendente | em_analise | atendida
imagens	JSONField	Lista de URLs (Cloudinary)
observacoes	TextField	Anotações da autoridade
score_prioridade	Float	Score calculado automaticamente
created_at / updated_at	DateTimeField	Timestamps automáticos

Alerta

Alertas emitidos pelas autoridades para a população de uma área geográfica.

Recurso

Catálogo de recursos de combate (viaturas, bombas, fontes d'água) com localização.

Verificacao

Registro de visita de campo: autoridade vincula verificação a uma ocorrência com resultado e fotos.

6. Autenticação

A API utiliza **JWT (JSON Web Token)** via biblioteca SimpleJWT. Todas as rotas (exceto login e registro) exigem o header:

```
Authorization: Bearer <access_token>
```

O access token expira em **8 horas**. O refresh token permite obter novos access tokens sem novo login. Perfis de acesso:

- **cidadao** — pode criar ocorrências e ler as próprias.
- **autoridade** — lê todas as ocorrências, atualiza status, emite alertas e gerencia recursos.

7. Endpoints da API

Base URL: **http://<host>:8000/api/**

7.1 Contas e Autenticação — /api/accounts/

Método	Endpoint	Descrição
POST	/accounts/login/	Login — retorna access_token e refresh_token
POST	/accounts/token/refresh/	Renova access_token com refresh_token válido
POST	/accounts/register/	Cadastro de novo usuário (nome, email, senha, role)
GET	/accounts/me/	Dados do usuário autenticado
PATCH	/accounts/me/	Atualizar perfil (nome, fcm_token)
POST	/accounts/logout/	Invalida o refresh token

7.2 Ocorrências — /api/ocorrencias/

Método	Endpoint	Descrição
GET	/ocorrencias/	Lista ocorrências (autoridade: todas; cidadão: só as suas)
POST	/ocorrencias/	Cria nova ocorrência — score calculado automaticamente
GET	/ocorrencias/{id}/	Detalhe de uma ocorrência
PATCH	/ocorrencias/{id}/	Atualiza status e observações (somente autoridade)
GET	/ocorrencias/{id}/score/	Retorna score de prioridade e recomendação
POST	/ocorrencias/detectar-duplicata/	Verifica se há ocorrência similar nas últimas 24h em raio de 500m

7.3 Alertas — /api/alertas/

Método	Endpoint	Descrição
GET	/alertas/	Lista alertas emitidos pelas autoridades

Método	Endpoint	Descrição
POST	/alertas/	Emite novo alerta para uma área (somente autoridade)
GET	/alertas/{id}/	Detalhe de um alerta

7.4 Recursos — /api/recursos/

Método	Endpoint	Descrição
GET	/recursos/	Lista recursos de combate cadastrados
POST	/recursos/	Cadastra novo recurso (somente autoridade)
GET/PATCH/DELETE	/recursos/{id}/	Detalhe, edição e remoção de recurso

7.5 Verificações — /api/verificacoes/

Método	Endpoint	Descrição
GET	/verificacoes/	Lista verificações de campo
POST	/verificacoes/	Registra verificação vinculada a uma ocorrência
GET	/verificacoes/{id}/	Detalhe da verificação

7.6 Exemplos de Requisição

Login:

```
POST /api/accounts/login/ Content-Type: application/json { "email": "cidadao@vcf.test", "password": "Cidadao@123" }
```

Criar ocorrência:

```
POST /api/ocorrencias/ Authorization: Bearer Content-Type: application/json { "anonima": false, "latitude": -8.1234567, "longitude": -41.1234567, "descricao": "Fogo se espalhando pela caatinga próximo à BR-407", "nivel_fogo": "alto", "nivel_fumaca": "Fumaça densa", "nivel_risco": "Próximo a residência", "ha_fumaca": true, "proximo_residencia": true, "imagens": [] }
```

8. WebSocket (Tempo Real)

A API utiliza **Django Channels** para comunicação bidirecional em tempo real. O WebSocket requer servidor ASGI (Daphne) e Redis configurado.

Endpoints WebSocket:

- **ws://<host>/ws/reports/?token=<jwt>** — Stream de novas ocorrências para autoridades.
- **ws://<host>/ws/user/<id>/** — Atualizações de status para o cidadão autor.

Nota: Em ambiente de desenvolvimento com *manage.py runserver*, o WebSocket retorna 404 pois o servidor WSGI não suporta conexões assíncronas. Use Daphne em produção:

```
daphne -b 0.0.0.0 -p 8000 semiaridofogo.asgi:application
```

9. Notificações Push (FCM)

A cada nova ocorrência recebida, a API envia automaticamente uma notificação push via **Firebase Cloud Messaging (FCM)** para todos os dispositivos de autoridade que possuem *fcm_token* cadastrado.

O campo **fcm_token** é atualizado pelo app Flutter no login e quando o sistema de notificações do dispositivo gera um novo token.

10. Variáveis de Ambiente

Variável	Descrição
SECRET_KEY	Chave secreta Django (obrigatório em produção)
DEBUG	True em desenvolvimento, False em produção
ALLOWED_HOSTS	Hosts permitidos (ex: localhost,127.0.0.1,domínio.com)
DATABASE_URL	URL do banco (sqlite:///db.sqlite3 ou postgresql://...)
FCM_SERVER_KEY	Chave do servidor Firebase para push notifications
CLOUDINARY_CLOUD_NAME	Nome do cloud Cloudinary para upload de imagens
CLOUDINARY_API_KEY	Chave de API Cloudinary
CLOUDINARY_API_SECRET	Secret Cloudinary
REDIS_URL	URL do Redis para Django Channels (ex: redis://localhost:6379)

11. Instalação e Execução

```
# 1. Clone o repositório (branch master)
git clone https://github.com/igortawan/API-Voz-conta-o-fogo.git
# 2. Crie e ative o ambiente virtual
```

```
python -m venv venv venv\Scripts\activate # Windows
# 3. Instale as dependências
pip install -r requirements.txt
# 4. Configure o .env (copie .env.example e preencha)
copy .env.example .env
# 5. Execute as migrações
python manage.py migrate
# 6. Crie um superusuário (autoridade)
python manage.py createsuperuser
# 7. Inicie o servidor
python manage.py runserver 0.0.0.0:8000
```

12. Uso de Inteligência Artificial no Desenvolvimento

Durante o desenvolvimento deste projeto, utilizamos **Inteligência Artificial como ferramenta de orientação técnica**. A IA auxiliou nas decisões de arquitetura, na explicação de conceitos, na revisão de abordagens e na sugestão de passos a seguir.

A **escrita do código, as decisões de negócio e a lógica da aplicação foram realizadas pela equipe de desenvolvimento**. A IA não gerou o código de forma autônoma — seu papel foi o de um consultor técnico que orienta, sugere e explica, cabendo à equipe avaliar, adaptar e implementar cada solução.